**สาขาวิชาวิศวกรรมคอมพิวเตอร์ มหาวิทยาลัยเทคโนโลยีสุรนารี**

**วิชา ENG23 2011 ระบบฐานข้อมูล (Database Systems)**

## ปฏิบัติการที่ 7 MySQL Stored Procedures and MySQL Stored Function

**<u>จุดประสงค์การปฏิบัติการ</u>**

- นักศึกษาสามารถเขียน Stored Procedures ได้
- นักศึกษาสามารถเขียน Stored Function ได้

**<u>ไฟล์ที่จำเป็น</u> :** https://bit.ly/3cp72IA

### แบบทดสอบที่ 7.1

- (2 คะแนน) ให้นักศึกษาสร้าง Stored Procedure ชื่อว่า “getAccountCustomer” เพื่อเรียกดูข้อมูลทั้งหมดจากตาราง Account, ตาราง Depositor และตาราง Customer ดังตัวอย่าง

Result Grid  Filter Rows: Search  Export:

| account_num... | branch_name | balance | customer_name | account_numb... | customer_name | customer_st... | customer_city |
|---|---|---|---|---|---|---|---|
| 1 | SUT | 150 | Som | 1 | Som | Mitr-Pab | Korat |
| 3 | Mall | 200 | Dang | 3 | Dang | Univ | Korat |

- (2 คะแนน) ให้นักศึกษาสร้าง Stored Procedure ชื่อว่า “getTotalAsset” เพื่อเรียกดูผลรวมของจำนวนสินทรัพย์ (Asset) จากธนาคารทุก ๆ สาขา (branch) มีอยู่ในระบบ ดังตัวอย่าง

Result Grid  Filter Rows: Search  Export:

| totalAsset |
|---|
| 3000000 |

- (2 คะแนน) ให้นักศึกษาสร้าง Stored Procedure ชื่อว่า “getAssetAvgBalance” เพื่อเรียกดูสินทรัพย์ (Asset) และค่าเฉลี่ยนของจำนวนเงินในบัญชี (balance) ของลูกค้าทั้งหมดของธนาคารสาขาที่ผู้ใช้งานระบุเข้าไป ดังตัวอย่าง

Limit to 1000 rows

```
1 •  CALL `db_lab7`.`getAssetAvgBalance`('Mall', @b_asset, @avg_a_balance);
2 •  Select @b_asset, @avg_a_balance;
```

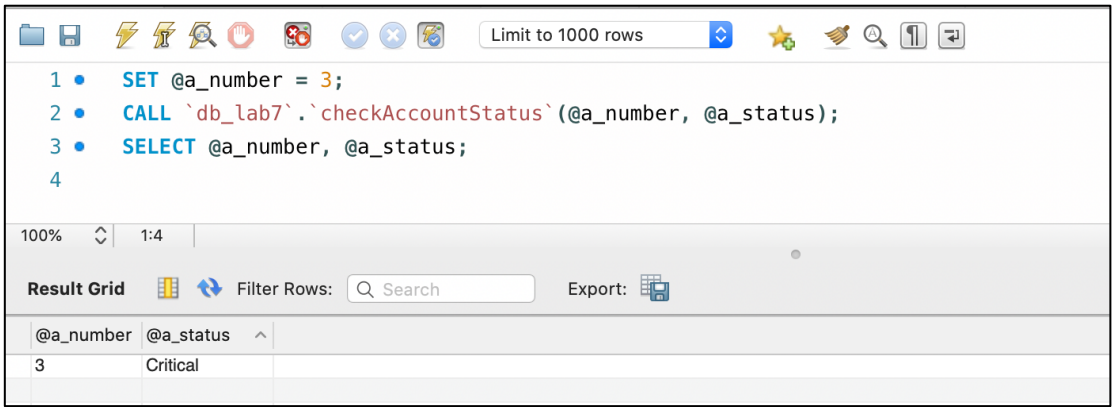
100%  28:1

Result Grid  Filter Rows: Search  Export:

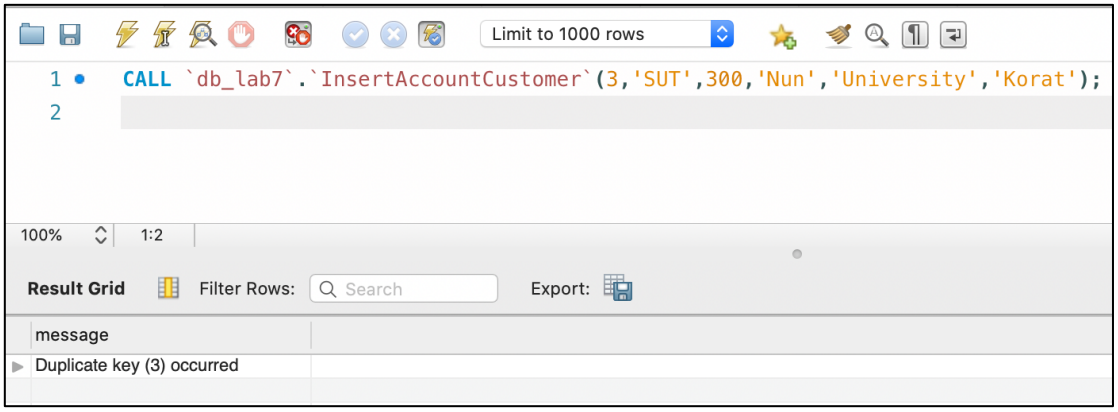
| @b_asset | @avg_a_balance |
|---|---|
| 1000000 | 200 |

- (2 คะแนน) ให้นักศึกษาสร้าง Stored Procedure ชื่อว่า “checkAccountStatus” เพื่อเรียกดูตามหมายเลขบัญชีลูกค้าธนาคาร (account_number) ว่าลูกค้าคนไหนมีสถานะการเงินเป็นอย่างไร โดยมีเงื่อนไขดังนี้
    - ถ้าบัญชีลูกค้าธนาคารมีเงินในบัญชี (balance) มากกว่าผลรวมจำนวนเงินกู้ (loan) ให้ Status = ‘OK’
    - ถ้าบัญชีลูกค้าธนาคารมีเงินในบัญชี (balance) เท่ากับผลรวมจำนวนเงินกู้ (loan) ให้ Status = ‘Warning’
    - ถ้าบัญชีลูกค้าธนาคารมีเงินในบัญชี (balance) น้อยกว่าผลรวมจำนวนเงินกู้ (loan) ให้ Status = ‘Critical’

  ดังตัวอย่าง




- (2 คะแนน) ให้นักศึกษาสร้าง Stored Procedure ชื่อว่า “InsertAccountCustomer” เพื่อเพิ่มข้อมูลเข้าไปในตาราง Account, Depositor และ Customer โดยจะต้องตรวจสอบความซ้ำซ้อนของคีย์หลัก (Primary Key) ด้วย ดังตัวอย่าง



**แบบทดสอบที่ 7.2**

- (5 คะแนน) ให้นักศึกษาสร้าง Stored Function ชื่อว่า “GenAccountNumber” เพื่อเปลี่ยนหมายเลขบัญชีที่จะเพิ่มเข้ามาในฐานข้อมูล โดยให้บวก 100 จากรหัสเดิมที่กรอกเข้ามา และส่งหมายเลขบัญชีใหม่กลับคืนไป ดังตัวอย่าง

```
INSERT INTO account VALUES(GenAccountNumber(4), 'SUT', 3000);
```

| account_number | branch_name | balance |
|---|---|---|
| 1 | SUT | 150 |
| 3 | Mall | 200 |
| 104 | SUT | 3000 |
| NULL | NULL | NULL |

- (5 คะแนน) ให้นักศึกษาสร้าง Stored Function ชื่อว่า “BranchNameToID” เพื่อเปลี่ยนจากชื่อสาขา (branch_name) ไปเป็นรหัส โดยกำหนดให้
  - สาขา SUT รหัส 0001
  - สาขา Mall รหัส 0002

  ดังตัวอย่าง

```
SELECT
    BranchNameToID(branch_name),
    branch_name,
    branch_city,
    asset
FROM
    branch;
```

| Branc... | branch_name | branch_city | asset |
|---|---|---|---|
| 0001 | SUT | Korat | 2000000 |
| 0002 | Mall | Korat | 1000000 |